Índice

1. Vectores de $\mathbb{R}^n$ - V01 **2**
1.1. Escena 1 - Definición de vector de $\mathbb{R}^n$ 2
1.2. Escena 2 - Notación vectores de $\mathbb{R}^n$ 3
1.3. Escena 3 - Selección de elementos de un vector de $\mathbb{R}^n$ 5

2. Trozos comunes de código **7**
2.1. Carga de la librería Manim y NacAL 7
2.2. Creditos 8

3. Rodando: 1,2,3... ąacción! **8**

Lección 1 del curso - Vídeo 1

Marcos Bujosa

27 de enero de 2026

1. Vectores de $\mathbb{R}^n$ - V01

1.1. Escena 1 - Definición de vector de $\mathbb{R}^n$

```
«Carga de la librería Manim y NacAL con AzureService»

«copyright»

class L01_V01_E01_VectoresDefinicion(VoiceoverScene):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )

        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
        myTemplate.add_to_preamble(r"\usepackage{nacal}")

        # Copyright lateral
        «copyrightLateral»

        # Portada
        titulo = Title(r"Definición de vector de \R[n] y notación",
                    tex_template = myTemplate,
                    font_size=70).set_color(BLUE)
        self.play(Write(titulo))
        self.wait(1.5)
        self.play(FadeOut(titulo))

        # Definición de vector
        definicion = Tex("Un vector de ",
                         r"$\R[n]$",
                         r" es una \emph{lista ordenada} de $n$ números",
                         tex_template = myTemplate,
                         ).to_edge(UP).set_width(13)
        with self.voiceover(text=r"Un vector de R n es una lista ordenada de números. ") as tracker:
            self.add(definicion)

        # Aclaración de la notación
        with self.voiceover(text="La R indica que los números son reales.") as tracker:
            self.play(Circumscribe(definicion[1][0], fade_out=True), run_time=tracker.duration)

        with self.voiceover(text="""Y el superíndice n indica que la lista contiene n
        números.""") as tracker:
            self.play(Circumscribe(definicion[1][1], fade_out=True), run_time=tracker.duration)
            self.wait(0.3)

        with self.voiceover(text="""que la lista sea ordenada significa que importa el orden en el
        que aparecen sus elementos.""") as tracker:
            self.play(Circumscribe(definicion[2][5:18], fade_out=True), run_time=tracker.duration)

        # Ejemplos
        Ej = Tex(r"\textbf{Ejemplos:}",
                 tex_template = myTemplate,
```

```
            font_size=50).set_color(GREEN).next_to(definicion, DOWN, aligned_edge=LEFT)
        self.add(Ej)

        d = nc.Vector([1,2,3],'h')
        Ej1 = MathTex( d.latex(), "\\ne", nc.Vector(reversed(d),'h').latex() )
        with self.voiceover(text="""Por ejemplo, los vectores, uno dos tres y tres dos uno, son
        distintos.""") as tracker:
            self.play(FadeIn(Ej1))
            self.wait(tracker.duration-0.5)
            self.play(FadeOut(Ej1))

        p = nc.Vector([sp.pi,sp.pi,sp.pi,sp.pi],'h')
        Ej2 = MathTex(p.latex(), r"\ne", nc.Vector(p[(1,2),'h').latex() )
        with self.voiceover(text=r"""También son distintos los vectores con distinta cantidad de
        elementos. Por ejemplo, el vector de la izquierda pertenece a R 4 por ser una lista de
        4 números. El de la derecha pertenece a R
        2.""") as tracker:
            self.add(Ej2)
            self.wait(tracker.duration-0.5)
            self.play(FadeOut(Ej2))

        c = nc.Vector([sp.pi,nc.fracc(3,4),0,0.11],'h')
        Ej3 = MathTex(c.latex(),"=",c.latex())
        with self.voiceover(text=r"""En consecuencia, dos vectores serán iguales si, y solo si, sus
        correspondientes listas son idénticas""") as tracker:
            self.play(FadeIn(Ej3))
            self.wait(tracker.duration)
            self.play(FadeOut(Ej3))
        self.play(FadeOut(Ej))
        self.wait()
```

1.2. Escena 2 - Notación vectores de $\mathbb{R}^n$

```
class L01_V01_E02_VectoresNotacion(VoiceoverScene):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )

        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
        myTemplate.add_to_preamble(r"\usepackage{nacal}")

        # Copyright lateral
        «copyrightLateral»

        # Definición de vector
        definicion = Tex("Un vector de ",
                         r"$\R[n]$",
                         r" es una \emph{lista ordenada} de $n$ números",
                         tex_template = myTemplate,
                         ).to_edge(UP).set_width(13)
        self.add(definicion)
        d = nc.Vector([1,2,3],'h')
        p = nc.Vector([sp.pi,sp.pi,sp.pi,sp.pi],'h')

        # Notacion
        Notac = Tex(r"\textbf{Notación:}",
                 tex_template = myTemplate,
                 font_size=50).set_color(BLUE).next_to(definicion, DOWN, aligned_edge=LEFT)
        self.add(Notac)
        self.wait()

        a    = nc.Vector( [5, 1, 10] , 'h')
        Not1 = MathTex(a.latex(),        tex_template = myTemplate,)
        Not2 = MathTex(r"=",             tex_template = myTemplate,)
        Not3 = MathTex(a.copy().latex(), tex_template = myTemplate,)
```

```
grp1 = VGroup(Not1,Not2,Not3).arrange(RIGHT)
with self.voiceover(text=r"""Para expresar un vector basta indicar la lista de elementos en su
correspondiente orden.""") as tracker:
    self.add(grp1)
    self.play(Circumscribe(definicion[2][5:18]),run_time=tracker.duration)

with self.voiceover(text=r"""Por este motivo podemos escribir un mismo vector tanto en
horizontal como en vertical. Pero tenga en cuenta que la
mayoría de manuales no respetan este convenio, y consideran,
al contrario de lo que haremos nosotros, que vectores fila y
vectores columna son objetos distintos.""") as tracker:
    self.wait(tracker.duration+0.3)

with self.voiceover(text=r"""Siempre escribiremos la lista de números encerrada entre
paréntesis; poniendo una coma detrás de cada elemento cuando
escribamos el vector en horizontal.""") as tracker:
    self.play(Indicate(grp1[0][0][::len(grp1[0][0])-1]),
              Indicate(grp1[2][0][0:2]), Indicate(grp1[2][0][-2:]),
              run_time=tracker.duration/2)
    self.play(Flash(grp1[0][0][2]),
              Flash(grp1[0][0][4]), Flash(grp1[0][0][7]),
              run_time=tracker.duration/8)
    self.wait(tracker.duration/8)
    self.play(Circumscribe(grp1[0]))
    self.play(FadeOut(grp1))

VectorNoNumero =  MathTex(r"(3)",
                          r"\ne",(3*nc.V1(1)).latex(),
                          r"\in\R[1]",
                          tex_template = myTemplate,)
with self.voiceover(text=r"""Así podremos distinguir un número entre paréntesis de un
vector de R 1;""") as tracker:
    self.add(VectorNoNumero)
    self.play(Indicate(VectorNoNumero[0]),run_time=tracker.duration*2/3)
    self.play(Indicate(VectorNoNumero[2]),run_time=tracker.duration/3)
with self.voiceover(text=r"""es decir, de una lista con un solo número.""") as tracker:
    self.play(Indicate(VectorNoNumero[3]),
              Flash(definicion[2][-9]),
              run_time=tracker.duration)
    self.play(FadeOut(VectorNoNumero))

Vectores = MathTex(r"\Vect{a}, \Vect{b}, \Vect{c},\ldots\Vect{x}, \Vect{y}, \Vect{z}",
                   tex_template = myTemplate,).move_to( UP )
Vector1  = MathTex(r"\Vect{a}=",a.copy().latex(),   tex_template = myTemplate,)
Vector2  = MathTex(r"\Vect{d}=",d.copy().latex(),   tex_template = myTemplate,)
Vector3  = MathTex(r"\Vect{x}=",p.copy().latex(),   tex_template = myTemplate,)
grp3 = VGroup(Vector1,Vector2,Vector3).arrange(RIGHT, buff=2).next_to(Vectores, DOWN)
with self.voiceover(text=r"""Para denotar vectores usaremos letras minúsculas en negrita
cursiva.""") as tracker:
    self.add(Vectores)
    self.add(grp3)
    self.wait(tracker.duration/2)
    self.play(Indicate(Vectores),run_time=tracker.duration/2)
    self.play(FadeOut(Vectores))
    self.play(Indicate(Vector1[0][0],scale_factor=2.),
              Indicate(Vector2[0][0],scale_factor=2.),
              Indicate(Vector3[0][0],scale_factor=2.),
              run_time=1.5)
    self.play(FadeOut(grp3))

Vnulo = MathTex(r"\Vect{0}", tex_template = myTemplate,)#.move_to( UP )
with self.voiceover(text=r"""Un cero en negrita denota un vector cuyas componentes son todas
nulas.""") as tracker:
    self.add(Vnulo)
    self.play(Indicate(Vnulo))
    self.wait(tracker.duration/2)
    self.play(FadeOut(Vnulo))
```

```
        Vnulo1 = MathTex(r"\Vect{0}=", nc.V0(1).latex(), ",",  tex_template = myTemplate,)
        Vnulo2 = MathTex(r"\Vect{0}=", nc.V0(2).latex(), ",",  tex_template = myTemplate,)
        Vnulo3 = MathTex(r"\Vect{0}=", nc.V0(3).latex(), ",",  tex_template = myTemplate,)
        Vnulo6 = MathTex(r"\Vect{0}=", nc.V0(6).latex(), ",",  tex_template = myTemplate,)
        VnuloN = MathTex(r"\Vect{0}\in\R[100]",             tex_template = myTemplate,)
        grp2   = VGroup(Vnulo1,Vnulo2,Vnulo3,Vnulo6,VnuloN).arrange(RIGHT, buff=0.7)
        with self.voiceover( text = r"""Fíjese que un cero en negrita no indica su número de
        componentes. Normalmente la cantidad de ceros se deduce del contexto.""" ) as tracker:
            self.add(grp2)
            self.wait(tracker.duration)
            self.play(FadeOut(grp2),FadeOut(Notac),FadeOut(definicion))
            self.wait(1.5)
```

1.3. Escena 3 - Selección de elementos de un vector de $\mathbb{R}^n$

```
class L01_V01_E03_VectoresElementos(VoiceoverScene):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )
        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
        myTemplate.add_to_preamble(r"""\usepackage{nacal} """)

        # Copyright lateral
        «copyrightLateral»

        # Notacion
        Notac = Tex(r"\textbf{Notación para los elementos:}",
                 tex_template = myTemplate,
                 font_size=50).set_color(BLUE).to_corner(UL)
        self.wait()
        self.add(Notac)
        self.wait()

        # Elementos de un vector
        v_generico  = nc.Vector(sp.symbols('a:5')[1:],'h')
        cs = MathTex(r"\Vect{a}=",
                     v_generico.latex(),
                     tex_template = myTemplate,)

        with self.voiceover(text = r"""Lo habitual es denotar cada elemento de un vector con la letra
        de su nombre sin negrita.""" ) as tracker:
            self.wait()
            self.play(FadeIn(cs), run_time=0.5)
            self.play( Circumscribe(cs[1][1]),
                       Circumscribe(cs[1][4]),
                       Circumscribe(cs[1][7]),
                       Circumscribe(cs[1][10]),
                       run_time=tracker.duration/2)

        with self.voiceover(text = r"""indicando con un subíndice su posición en la
        lista.""" ) as tracker:
            self.play( Flash(cs[1][2]),
                       Flash(cs[1][5]),
                       Flash(cs[1][8]),
                       Flash(cs[1][11]),
                       run_time=tracker.duration)
            self.play(FadeOut(cs))

        c = nc.Vector([sp.pi,nc.fracc(3,4),0,0.11],'h')
        vector_c = MathTex(r"\Vect{c}=",c.latex(),tex_template = myTemplate,)
        A = VGroup(*[ MathTex("c_"+str(i+1)+"=&"+sp.latex(e))  for i,e in enumerate(c.lista)
                    ]).arrange(DOWN,aligned_edge=LEFT, buff=.5)
        B = Brace(A, LEFT)
        C = VGroup(A,B)
        Elementos_c   = VGroup(vector_c, C).arrange(RIGHT, buff=1)
```

```
with self.voiceover(text = r"""Así, para el vector C """) as tracker:
    self.play(FadeIn(vector_c))
    self.play(GrowFromCenter(B),FadeIn(A))

with self.voiceover(text = r"""con c 1 denotamos su primera componente""") as tracker:
    self.play( Indicate(vector_c[1][1]),    Indicate(A[0]) )
with self.voiceover(text = r"""con c 2 la segunda""") as tracker:
    self.play( Indicate(vector_c[1][3:6]),  Indicate(A[1]) )
with self.voiceover(text = r"""y del mismo modo con el resto de componentes""") as tracker:
    self.play( Indicate(vector_c[1][7],
                        run_time=tracker.duration/2),
               Indicate(A[2],
                        run_time=tracker.duration/2) )
    self.play( Indicate(vector_c[1][9:13],
                        run_time=tracker.duration/2),
               Indicate(A[3],
                        run_time=tracker.duration/2) )
    self.wait(0.5)
    self.play( FadeOut(vector_c), FadeOut(B), FadeOut(A) )
    self.wait(0.5)

with self.voiceover(text = r"""El hecho de emplear dos tipos de fuentes:""" ) as tracker:
    self.add(cs)
    self.wait(tracker.duration)

with self.voiceover(text = r"""con negrita los vectores y sin negrita los componentes,
dificulta distinguirlos a primera vista""" ) as tracker:
    self.play( Indicate(cs[0][ 0],scale_factor=2.),
               Indicate(cs[0][ 0],scale_factor=2.),
               Indicate(cs[1][ 1],scale_factor=2.),
               Indicate(cs[1][ 4],scale_factor=2.),
               Indicate(cs[1][ 7],scale_factor=2.),
               Indicate(cs[1][10],scale_factor=2.), run_time=tracker.duration*2/3)

MTa = MathTex(r"\eleVR{a}{i}",tex_template = myTemplate).scale(3)
MTb = MathTex(r"{a}_{i}=",tex_template = myTemplate).scale(3).next_to(MTa, LEFT)
VG  = VGroup(MTb,MTa)
with self.voiceover(text = r"""Es más clara y operativa una notación que use un único tipo de
fuente, y que denote la selección de elementos con un operador. Por ejemplo con una barra
vertical.""" ) as tracker:
    self.play(cs.animate.to_corner(DL),
              run_time=tracker.duration*4/5)
    self.play(Indicate(VG[1][0][1]))
    self.wait(0.5)

def VectorGenerico(s,n):
    elem = lambda s,i: sp.Symbol(r'\eleVR{'+ s +'}{'+ str(i) + '}')
    return nc.Vector([elem(s,i) for i in range(1,n+1)], 'h')

v_generico2 = VectorGenerico('a',4)
cs2 = MathTex(r"=",
              v_generico2.latex(),
              tex_template = myTemplate,).next_to(cs, RIGHT)

VGB = VGroup(*[MathTex(sp.latex(e) + "=\; & \eleVR{a}{" + str(i) + "}",
                       tex_template = myTemplate)
               for i,e in enumerate(v_generico.lista,1)
               ]).scale(3)

with self.voiceover( text = r"""Por ello, para denotar una componente, escribiremos un
subíndice con una barra que medie entre el vector y el índice de la
componente""" ) as tracker:
    self.add(VG[1])
    self.wait(tracker.duration/3)
    self.play(Indicate(VG[1][0][1:], run_time=tracker.duration/4))
    #self.wait(tracker.duration/3)
    self.play(Indicate(VG[1][0][-1], run_time=tracker.duration/4))
    self.play(Write(VG[0]))
```

```
        self.play(VG.animate.move_to([0,0,0]), run_time=tracker.duration/6)

        source0 = VG[0][0]
        source1 = VG[1][0]
        target  = VGB[0][0]

        transform_index0 = [[0,1,2], [0,1,2]]
        transform_index1 = [[0,1,2], [3,4,5]]

        self.play(
            *[
                ReplacementTransform(source0[i],target[j], rate_func=rate_functions.smooth)
                for i,j in zip(*transform_index0)
            ],
            *[
                ReplacementTransform(source1[i],target[j], rate_func=rate_functions.smooth)
                for i,j in zip(*transform_index1)
            ],
            run_time=tracker.duration/5)

        self.play(FadeIn(cs2) )
        self.play(FadeTransform(VGB[0][0][0:2],cs[1][ 1: 3]),
                  FadeTransform(VGB[0][0][3:],cs2[1][ 1: 4]),
                  run_time=.7)
        self.play(FadeIn(VGB[1]), FadeOut(VGB[0][0][2]))
        self.play(FadeTransform(VGB[1][0][0:2],cs[1] [4: 6]),
                  FadeTransform(VGB[1][0][3:],cs2[1][ 5: 8]),
                  FadeOut(VGB[1][0][2]),
                  run_time=.6)
        self.play(FadeIn(VGB[2]))
        self.play(FadeTransform(VGB[2][0][0:2],cs[1][ 7: 9]),
                  FadeTransform(VGB[2][0][3:],cs2[1][9:12]),
                  FadeOut(VGB[2][0][2]),
                  run_time=.5)
        self.play(FadeIn(VGB[3]))
        self.play(FadeTransform(VGB[3][0][0:2],cs[1][10:12]),
                  FadeTransform(VGB[3][0][3:],cs2[1][13:16]),
                  FadeOut(VGB[3][0][2]),
                  run_time=.4)
        self.play(FadeOut(Notac),FadeOut(cs),FadeOut(cs2))

    MTLR = MathTex(r"\eleVR{a}{i}",r"\;=\eleVL{a}{i}",tex_template = myTemplate).scale(3)
    with self.voiceover( text = r"""Además, admitiremos que el operador selector actúe tanto por
    la derecha como por la izquierda."""  ) as tracker:
        self.play(FadeIn(MTLR[0]), run_time=tracker.duration/2)
        self.play(FadeIn(MTLR[1]))
        self.wait(tracker.duration/3+0.5)
        self.play(FadeOut(MTLR))
        self.wait()
```

2. Trozos comunes de código

2.1. Carga de la librería Manim y NacAL

```
from manim import *
from manim_voiceover import VoiceoverScene
from manim_voiceover.services.gtts import GTTSService
import nacal as nc
import sympy as sp

# PARA LA TRADUCCIÓN (pero no me ha funcionado)

#from manim_voiceover.translate import get_gettext
# # It is good practice to get the LOCALE and DOMAIN from environment variables
```

```
#import os
#LOCALE = os.getenv("LOCALE")
#DOMAIN = os.getenv("DOMAIN")
# The following function uses LOCALE and DOMAIN to set the language, and
# returns a gettext function that is used to insert translations.
#_ = get_gettext()
```

```
from manim import *
from manim_voiceover import VoiceoverScene
from manim_voiceover.services.azure import AzureService
import nacal as nc
import sympy as sp

# PARA LA TRADUCCIÓN (pero no me ha funcionado)

#from manim_voiceover.translate import get_gettext
# # It is good practice to get the LOCALE and DOMAIN from environment variables
#import os
#LOCALE = os.getenv("LOCALE")
#DOMAIN = os.getenv("DOMAIN")
# The following function uses LOCALE and DOMAIN to set the language, and
# returns a gettext function that is used to insert translations.
#_ = get_gettext()
```

2.2. Creditos

```
copyright = Tex(r"\textcopyright{\;} 2024\; Marcos Bujosa  ")
CGG  = VGroup(copyright).rotate(PI/2).scale(0.5).to_edge(RIGHT, buff=0.1).set_color(GRAY_D)
self.add(CGG)
```

```
class ZCreditos(Scene):
    def construct(self):
        copyright = Tex(r"\textcopyright{\;} 2024 \; Marcos Bujosa")
        github = Tex(r"\texttt{https://github.com/mbujosab}").next_to(copyright, DOWN)
        CGG  = VGroup(copyright,github).scale(1.1)
        self.add(CGG)
        self.wait(10)
```

3. Rodando: 1,2,3... ąacción!

1. Generamos un fichero `mpeg` por cada escena

```
echo $escena | manim -pql $fichero.py --disable_caching
```

```
echo $escena | manim -qh $fichero.py --disable_caching
```

2. Concatenamos las escenas en un único fichero `mpeg` y añadimos música de fondo.

   - Montando la versión de baja resolución

```
rm -f $subdir/$video/$calidad/$video.mp4 list.txt
for f in $subdir/$video/$calidad/*.mp4 ; do echo file \'$f\' >> list.txt; done && ffmpeg -f concat -safe 0 -i list.txt -c

mkdir -p tmp

ffmpeg -i $subdir/$video/$calidad/$video.mp4 -i $music.mp3 -filter_complex "[0:a]apad[main]; [1:a]volume=0.04,apad[A]; [m
```

a) La primera parte de este script concatena todos los archivos de video `.mp4` encontrados en una carpeta específica en un solo archivo de video. Lo hace en dos pasos:

- **Genera un archivo de lista de archivos (`list.txt`):**
  - Utiliza un bucle `for` para iterar sobre todos los archivos con extensión `.mp4` en la carpeta especificada por la variable `$subdir/$video/$calidad/`.
  - Por cada archivo encontrado, agrega una línea al archivo `list.txt` que contiene la ruta completa del archivo.
  - Utiliza el operador `> >` para redirigir la salida al final del archivo `list.txt`.
- **Utiliza ffmpeg para concatenar los archivos de video listados en `list.txt`:**
  - Utiliza `ffmpeg` con la opción `-f concat` para indicar que se está utilizando un archivo de lista para concatenar.
  - `-safe 0` se usa para permitir la lectura de archivos de cualquier ruta.
  - `-i list.txt` especifica el archivo de lista generado anteriormente como entrada.
  - `-c copy` indica a `ffmpeg` que simplemente copie los flujos de audio y video de entrada sin realizar ninguna conversión.
  - Finalmente, el resultado se guarda en un nuevo archivo de video con el mismo nombre que la variable `$video`, pero con la extensión `.mp4`, en la carpeta especificada por `$subdir/$video/$calidad/`.

b) En la segunda parte el comando de 'ffmpeg' realiza varias acciones para procesar un video junto con un archivo de audio y generar un nuevo archivo de video:

1) **Entradas de archivos de video y audio:**
   - `-i $subdir/$video/$calidad/$video.mp4`: Especifica el archivo de video de entrada.
   - `-i $music.mp3`: Especifica el archivo de audio de entrada que se usará como música de fondo.
2) **Filtro de complejidad (`-filter_complex`):**
   - `[0:a]apad[main]`: Añade un silencio al audio del video de entrada (`[0:a]`) y lo asigna a la etiqueta `[main]`.
   - `[1:a]volume=0.04,apad[A]`: Ajusta el volumen del audio de entrada (`[1:a]`) al 4 % del volumen original, añade un silencio y lo asigna a la etiqueta `[A]`.
   - `[main][A]amerge[out]`: Fusiona los flujos de audio etiquetados como `[main]` y `[A]` en un solo flujo de audio, asignado a la etiqueta `[out]`.
3) **Configuración de códecs y opciones de salida:**
   - `-c:v libx264`: Utiliza el códec de video H.264 para codificar el video de salida.
   - `-c:a aac`: Utiliza el códec de audio AAC para codificar el audio de salida.
   - `-map 0:v -map "[out]"`: Mapea el flujo de video del archivo de video de entrada y el flujo de audio resultante del filtro complejo.
   - `-preset ultrafast`: Establece el preset de codificación en ültrafast"para una codificación más rápida.
   - `-threads 0`: Utiliza el número máximo de hilos disponibles para la codificación.
   - `-profile:v baseline`: Especifica el perfil de codificación de video como "baseline".
   - `-ac 2`: Establece el número de canales de audio a 2 (estéreo).

- `-pix_fmt yuv420p`: Especifica el formato de píxeles del video de salida como yuv420p.
- `-shortest`: Hace que la duración del video de salida sea igual a la duración del archivo de video de entrada más corto o la duración del audio de entrada más corto.
- `-y tmp/$video.mp4`: Especifica el nombre del archivo de video de salida y la ruta donde se guardará.

En resumen, este comando de `ffmpeg` combina un video con un archivo de audio de fondo, ajustando el volumen del audio de fondo y fusionando los flujos de audio para crear un nuevo archivo de video con las especificaciones dadas.

- Montando la versión de resolución HD1080

```
ffmpeg -i L01_V01_Vectores.mp4 voz.mp3
ffmpeg -y -i 3Blue1Brown-Zeta-X2.mp3 -i voz.mp3 -filter_complex "[0:a]volume=.15[A];[1:a]|
map [out] -c:a libmp3lame -q:a 4 voz_y_musica.mp3
ffmpeg -i L01_V01_Vectores.mp4 -an videomudo.mp4
ffmpeg  -i videomudo.mp4 -i voz_y_musica.mp3 -shortest final.mp4
```

```
ffmpeg -y -i $subdir/$video/$calidad/$video.mp4 tmp/voz.mp3
ffmpeg -y -i $music.mp3 -i tmp/voz.mp3 -filter_complex "[0:a]volume=.15[A];[1:a][A]amerge[out]" -map [out] -c:a libmp3lam
ffmpeg -y -i $subdir/$video/$calidad/$video.mp4 -an tmp/videomudo.mp4
ffmpeg -y -i tmp/videomudo.mp4 -i tmp/voz_y_musica.mp3 -shortest tmp/$video.mp4
```

```
rm -f $subdir/$video/$calidad/$video.mp4 list.txt
for f in $subdir/$video/$calidad/*.mp4 ; do echo file \'$f\' » list.txt; done && ffmpeg -f concat -safe 0 -i list.txt -c

mkdir -p tmp

ffmpeg -i $subdir/$video/$calidad/$video.mp4 -i $music.mp3 -filter_complex "[0:a]apad[main]; [1:a]volume=0.04,apad[A]; [m
```

3. Fundimos a negro los últimos segundos del vídeo (y la música).

```
dur=$(ffprobe -loglevel error -show_entries format=duration -of default=nk=1:nw=1 "tmp/$video.mp4") && offset=$(bc -l <<
```

Este código es un script que utiliza las herramientas `ffprobe` y `ffmpeg` para aplicar efectos de fundido a un video.Aquí está desglosado paso a paso:

a) `dur=$(ffprobe -loglevel error -show_entries format=duration -of default=nk=1:nw=1 "tmp/$video.mp4")`: Este comando utiliza `ffprobe` para obtener la duración del video especificado por la variable `$video`. La duración se guarda en la variable `dur`.

b) `offset=$(bc -l \<\<\<"$dur7)`: Luego, se calcula un valor de desplazamiento (`offset`) restando 7 segundos de la duración del video. Esto sugiere que el objetivo es aplicar los efectos de fundido durante los últimos 7 segundos del video.

c) `ffmpeg -i "tmp/$video.mp4filter_complex "[0:v]fade=type=in:duration=0,fade=type=out:du time='$offset'[v];[0:a]afade=type=in:duration=1,afade=type=out:duration=6:start_- time='$offset'[a]map "[v]map "[a]y $video.mp4`: Finalmente, se utiliza `ffmpeg` para aplicar los efectos de fundido. El filtro `-filter_complex` se utiliza para aplicar un

fundido de entrada (`fade=type=in`) y un fundido de salida (`fade=type=out`) tanto al video como al audio. El fundido de entrada del audio (`afade=type=in:duration=1`) tiene una duración de 1 segundo. Los fundidos de salida de audio y video tienen una duración de 6 segundos y comienzan en el tiempo especificado por el `offset`. El resultado se guarda en un nuevo archivo de video con el nombre `$video.mp4`.

En resumen, este script toma un video, calcula su duración, y luego aplica efectos de fundido de entrada y salida tanto al video como al audio, comenzando 7 segundos antes del final del video.

4. Copiamos el resultado a un lugar público

```
cp -f $video.mp4 $subdir/$video.mp4
```